

Lab 02 - Frames, Formats and Aspect Ratios for Film Delivery

Course: Create Short Video Film using AI (TGS-2020505925) | Version v12.0 | 27 September 2026 Topic 02: AI Image Processing, Generation and Visual Enhancement Outcome: ELO2 | In-class time: 15 minutes of scheduled practical time

Competency	Statement
Ability A2	Apply the principles of processing, filtering and analysis methods for video data
Knowledge K3	Methods to represent image and video data

Scenario

Before any shot of The Last Ferry is generated, the team must agree the master format and the delivery ladder. You inspect a keyframe and the 12-second rough cut the way the machine sees them - arrays, channel order, frame rate, duration and bitrate - and measure what a 9:16, 4:5 or 1:1 reframe keeps of the 16:9 master.

What you will produce

A clip report (shape, dtype, BGR/RGB/HSV of the calibration chips, fps, frame count, duration, bitrate, compression ratio), a contact sheet, and a reframe per delivery format with the percentage of the master each one keeps.

Tools: Python 3 | opencv-python | NumPy | no network

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: opencv-python>=4.10, numpy.

API currency. Everything taught here runs on the base opencv-python wheel plus NumPy - no contrib modules and no downloaded model weights. SSIM is implemented in NumPy because cv2.quality is contrib-only. Generative steps are optional: every lab supplies SIMULATED reference media so it can be completed offline, and asks you to record the model ID and date if you generate your own.

Workflow

1. Read the keyframe as an array
2. Verify channel order with the colour chips
3. Measure the clip: fps, frames, duration, bitrate
4. Reframe for each delivery format

5. Decide the master format and safe area

Files in this folder

Path	What it is
reference/keyframe-shot2.png	Keyframe of shot S2 with four exact colour chips bottom-left (SIMULATED)
reference/rough-cut.mp4	12-second 960x540 24 fps rough cut: three shots (SIMULATED clip)
data/delivery-specs.json	Master and four delivery formats with UI safe areas
inspect_clip.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom illustration of *The Last Ferry*, rendered deterministically with OpenCV and labelled on the frame, or an AI-generated reference copied unchanged with a PROVENANCE.md beside it. Nothing in this folder may be relabelled: a simulated stand-in is never presented as model output or as a photograph.

Steps

1. Install the environment once (see Environment) and run everything from inside this lab folder.
2. Load the keyframe and print its shape and dtype. Expect (540, 960, 3) uint8: height first, then width, then three channels.

```
python3 -c "import cv2;im=cv2.imread('reference/keyframe-shot2.png');print(im.shape, im.dtype)"
```

3. cv2.imread returns None on a bad path instead of raising - guard it. Try a wrong filename and confirm you get None, not an exception.

```
python3 -c "import cv2;print(cv2.imread('reference/nope.png'))"
```

4. Read the first colour chip at x=21, y=515. Arrays are indexed [row, column] = [y, x]. The chip is pure blue: OpenCV reports BGR [255, 0, 0].

```
python3 -c "import cv2;im=cv2.imread('reference/keyframe-shot2.png');print(im[515,21])"
```

5. Run the inspector. It prints each chip in BGR, RGB and HSV, measures the clip and writes the delivery reframes.

```
python3 inspect_clip.py --still reference/keyframe-shot2.png --clip reference/rough-cut.mp4 --specs data/delivery-specs.json --out out
```

6. Check the clock: 288 frames ÷ 24 fps = 12.0 s. Compare frames_declared with frames_counted - containers can misreport, so count when it matters.
7. Compute the uncompressed rate by hand: 960 × 540 × 3 × 24 = 37.3 MB/s. At the production master of 1920 × 1080 it is 149.3 MB/s. Compare with the file's bitrate and note the compression ratio.
8. Open out/contact-sheet.png (one frame per second). Identify the three shots and the seconds where they cut.
9. Open each out/reframe_*.png. The yellow rectangle is the platform UI safe area. Record the percentage of the master each format keeps: 9:16 keeps about 31.7%.

10. Decide: will The Last Ferry be generated natively in 16:9 and cut down, or will the vertical version be generated natively? Write the rule in out/format-decision.md with the numbers.
11. Write the blocking rule for every shot prompt: keep faces, the RED BOX and all captions inside the centre third of a 16:9 frame so every cut-down keeps them.

Verify

The chip at (21, 515) reads BGR [255, 0, 0]; the clip reports 24 fps, 288 frames and 12.0 s; the 9:16 reframe keeps about 31.7% of the master.

Expected outputs

- out/clip-report.json
- out/contact-sheet.png
- out/reframe_youtube_16x9.png, reframe_reels_9x16.png, reframe_feed_4x5.png, reframe_square_1x1.png
- out/format-decision.md

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- [] Printed shape, dtype and the BGR/RGB/HSV values of all four chips
- [] Hand calculation of the uncompressed data rate and the measured compression ratio
- [] Table of the percentage of the master each delivery format keeps
- [] Written master-format and blocking decision for The Last Ferry

Troubleshooting

Symptom	What to do
imread returns None	Run from inside the lab folder or give the full path; check the extension.
Colours look swapped in matplotlib	OpenCV is BGR; convert with <code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code> before plotting.
frames_declared differs from frames_counted	Trust the counted value; some containers store an estimate.
VideoCapture cannot open the mp4	Reinstall opencv-python (it bundles FFmpeg); do not use opencv-python-headless without FFmpeg.

References used in this lab

- [S5] Tertiary Courses public registration page and LMS-TMS course record -
<https://www.tertiarycourses.com.sg/wsqa-create-short-video-film-using-ai.html> (checked 27 Sep 2026).

Registered title 'Create Short Video Film using AI', TGS code, the five delivery topics, the six learning outcomes, the course description (concept, storyline, script, characters, storyboard, text-to-image, image-to-video, text-to-video, cinematic shots, voiceover, music, sound effects, captions, final edit), 2-day/16-hour duration and target learners.

- [S15] Google AI for Developers - Gemini API image generation documentation -
<https://ai.google.dev/gemini-api/docs/image-generation> (verified 6 Sep 2026).

Native image models for text-to-image and conversational image editing; supported aspect ratios including 16:9, 9:16, 4:5 and 21:9; output sizes up to 4K by model; every generated image carries a SynthID watermark.

- [S16] Google AI for Developers - Veo video generation documentation -
<https://ai.google.dev/gemini-api/docs/veo> (verified 6 Sep 2026).

Veo 3.1 model family with native audio; 4, 6 or 8 second clips; 16:9 or 9:16; 720p, 1080p or 4k; up to three reference images on supported models; image-to-video; first and last frame control; clip extension; asynchronous long-running operation that the client polls.

- [S17] OpenCV 4.13 Python package and FFmpeg, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92); ffmpeg/ffprobe optional*.

Every lab command runs on the base opencv-python wheel plus NumPy: VideoCapture / VideoWriter, cvtColor, calcHist/compareHist, Canny, ORB_create + BFMatcher, HuMoments, inpaint, CLAHE, filter2D, fastNIMeansDenoisingColored, Haar cascades from cv2.data.harcascades, HOGDescriptor_getDefaultPeopleDetector, CamShift and calcOpticalFlowFarneback. SSIM is implemented in NumPy because cv2.quality is contrib-only. FFmpeg is used only when present, to mux audio into the final export.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v12.0.